智行合一

《新能源汽车电池预警平台》
项目设计报告

智行合一有限公司

目录

1. 用户认证模块（AUTH） 3
1.1 用户登录（AUTH-01） 3
1.2 用户注册（AUTH-02） 4
2. 电池监控模块（MON） 6
2.1 改进 Transformer 的预测状态展示（MON-01） 6
2.2 异常预警（MON-02） 11
3. 阈值调整模块（THR） 13
3.1 阈值可视化配置（THR-01） 13
3.2 阈值方案快速回滚（THR-02） 14
4. 电池状态模块（BS） 14
4.1 电池状态可视化与续航数据（BS-01、BS-02） 14
5. AI 咨询模块（AI） 15
5.1 智能对话咨询（AI-01） 15
6. 意见反馈模块（FB） 17
6.1 反馈提交（FB-01） 17

1. 用户认证模块（AUTH）

1.1 用户登录（AUTH-01）

提供新能源汽车电池预警平台的带动态粒子背景的科技感登录入口，支持手机号和密码验证。

1.1.1 账号密码登录（AUTH-01-01）

用户通过在前端输入账号和密码提交表单验证，加入了非空检查与手机号格式验证，通过加密传输登录凭证获取登录权限，若登录成功，则进入监控面板主界面，若登录失败，则前端发出登录失败的提示（用户名错误/密码错误）。

在代码的设计上，前端通过 login.html 的登录 Ajax 组件，提交关于用户输入的用户名和密码表单到后端 flask 的本地 api_login 接口来获取登录凭证。关键实现在于通过 AJAX 登录，转发到 Tomcat Servlet 已设计好的接口后，Tomcat Servlet 进行校验（此处转发到 Tomcat，而不采用 Flask 直接验证，是由于在安卓课程中已搭建好关于登录的服务端，调用接口即可实现），Tomcat 中对前端提交的表单与数据库中的比对，从而获取登录权限。

登录界面如图：

图 1 登录界面

关键 Tomcat 接口部分代码展示：

```python
# ---------------  AJAX-API：登录 / 注册  --------------- #
TOMCAT_BASE = 'http://localhost:8080/Android'

@app.post('/api/login')
def api_login():
    """前端 AJAX 登录：转发到 Tomcat Servlet 校验"""
    username = request.form.get('username')
    password = request.form.get('password')
    try:
        r = requests.post(
            url: f'{TOMCAT_BASE}/LoginServlet',
            data={'loginAccount': username, 'loginPassword': password},
            timeout=5
        )
        ok = r.text.strip().lower() in ('true', 'ok', '1')
    except requests.RequestException as e:
        return jsonify({'status': 'error', 'message': f'后端连接失败：{e}'}), 502

    if ok:
        session.permanent = True
        session['user'] = username
        return jsonify({'status': 'success'})
    return jsonify({'status': 'fail', 'message': '用户名或密码错误'})
```

图 2 登录接口部分代码

1.1.2 登录状态保持（AUTH-01-02）

用户在登录后，即使退出浏览器后重新访问该网站，也不需要再次登录一次。具体实现如下：使用 Flask 的 session 和@login_required 装饰器管理会话，同时，前端通过 localStorage 存储 token，从而实现登录状态的存储。同时在主界面中提供“退出登录”按钮，返回登录界面。

1.2 用户注册（AUTH-02）

用户若是没有账号与密码，可以点击注册按钮进行账号注册，将新建账号和密码存入数据库中。

1.2.1 手机号注册（AUTH-02-01）

在注册中包括手机号格式验证，密码强度校验以及确认密码，其中手机号验证采用正则表达式/^1[3-9]\d{9}$/（仅处理 13 到 19 开头的 11 位手机号码），由前端 validatePhone 函数实现。

注册界面如下：

图 3 注册界面

Flask 中注册接口部分代码如下：

```python
@app.post('/api/register')
def api_register():
    """前端 AJAX 注册：转发到 Tomcat Servlet 写入数据库"""
    account = request.form.get('account')
    password = request.form.get('password')
    try:
        r = requests.post(
            url: f'{TOMCAT_BASE}/RegisterServlet',
            data={'registerAccount': account, 'registerPassword': password},
            timeout=5
        )
        txt = r.text.strip().lower()
    except requests.RequestException as e:
        return jsonify({'status': 'error', 'message': f'后端连接失败：{e}'}), 502

    if txt in ('true', 'ok', '1'):
        return jsonify({'status': 'success'})
    if txt == 'exists':
        return jsonify({'status': 'fail', 'message': '账号已存在'})
    return jsonify({'status': 'fail', 'message': '注册失败'})
```

图 4 注册接口部分代码示意

1.2.2 密码强度校验

包括密码强度检验（小于 20 且大于 6）以及密码一致性校验。密码强度校验前端通过 minlength="6" maxlength="20"属性进行检验。传输使用加密 AJAX 通信，将账号和密码以表单的形式通过 api/register 接口转发到 Tomcat Servlet 写入数据库。

数据库中登录注册表如下：

	StaffID	Account	Password
▶	1	test_user	123456
	2	111	111
	7	test_user2	123456
	9	1111	1111
	12	13423276729	111111
	13	13232422222	123456
	14	11111	11111
	16	111111	111111
	18	121	121
	19	1212	1212
	20	123123	123123
	21	144	144
	22	143	143
	24	1213	1213
	25	13423276728	123456
*	NULL	NULL	NULL

图 5 登录注册表

2. 电池监控模块（MON）

2.1 改进 Transformer 的预测状态展示（MON-01）

用户鼠标双击页面后，后端启动改进 Transformer 模型，根据用户的历史状态数据进行预测，将预测后的电池状态存储入 SQL 数据库中，前端读取数据库中新预测的电池监控字段通过 Echarts 进行数据可视化。

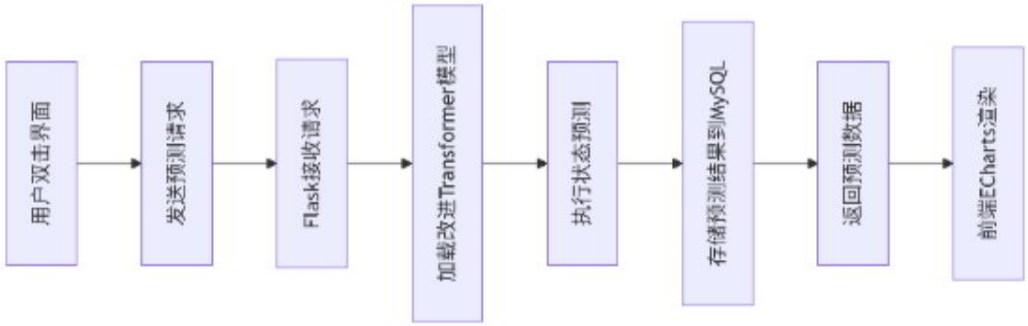

图 6 功能架构设计图

2.1.1 双击启动模型进行状态预测（MON-01-01）

该功能允许用户通过双击监控界面触发电池状态预测流程，系统将基于改进的 Transformer 模型对电池未来 2 小时内的电压、温度等电池监控字段进行预测，并将预测结果持久化存储至数据库，为后续可视化分析提供数据基础。

在代码实现上，前端监听全局双击事件（dblclick），当检测到用户在监控图表区域双击时，自动捕获当前电池 ID 和用户身份信息。前端通过 HTTPS 协议向/api/predict 接口发送预测请求，请求体包含用户认证令牌，后端服务验证 JWT

令牌有效性后，从数据库查询该电池预测的监测数据，为后续的可视化提供数据。

部分代码展示如下：

```
/* ---------- 双击启动（一次性） ---------- */
document.addEventListener('dblclick', () => {
  if (hasStarted) return;
  hasStarted = true;

  // 若有多个图表初始化函数可安全调用
  typeof startCombinedChart  === 'function' && startCombinedChart();
  typeof startCombinedChart2 === 'function' && startCombinedChart2();
  typeof startCombinedChart3 === 'function' && startCombinedChart3();
  typeof startCombinedChart4 === 'function' && startCombinedChart4();

  startDataUpload();  // ★ 真正开始定时展示
});

/* ---------- 首次页面加载 ---------- */
window.onload = () => {
  renderTable();
  generateFailureChart();           // 首次渲染
};
```

图 7 双击开启预测事件

加载融入元学习训练策略的改进 Transformer（基于 Pytorch 框架）进行时间序列的预测输出后，将预测结果写入 MySQL 数据库的 battery_metrics 表，每条记录包括预测的各电池字段，数据库设计图如下：

id	voltage1	voltage2	temperature1	temperature2	voltage_feature1	voltage_feature2	SDO_score	ICC	voltage_rate1	voltage_rate2	temperature_rate1	temperature_rate2	resistance1	res
569	3.43	3.63	35.89	32.78	0.35	0.31	0.0039	0.97	-0.6089	-0.4363	-12.04	-14.57	181.53	198
570	3.66	3.18	38.75	27.38	0.31	0.35	-0.0173	0.98	0.0712	-0.4461	-12.11	24.82	197.19	181
571	3.37	3.69	29.09	28.13	0.33	0.31	0.0113	0.99	0.6747	-0.179	28.26	-8.47	189.58	188
572	3.35	3.17	26.47	34.23	0.28	0.3	-0.0336	0.98	-0.1648	0.5936	15.49	-6.28	195.21	184
573	3.07	3.53	25.01	25.34	0.29	0.33	0.0028	0.99	0.0789	-0.1835	-8.76	5.87	194.79	184
574	3.13	3.14	36.82	31.45	0.3	0.3	-0.0029	0.98	0.1544	0.3972	6.21	16.58	194.75	198
575	3.43	3.22	20.13	29.79	0.3	0.33	-0.0022	0.97	0.2141	0.157	-29.61	-11.11	185.87	199
576	3.31	3.01	34.87	25.77	0.28	0.33	0.0069	0.99	0.4623	-0.3697	-27.22	29.96	185.57	187
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

图 8 预测数据表示意图

2.1.2 绘制电池各预测监控字段曲线图（MON-01-02）

前端系统采用现代化技术栈（html5），并基于模块化与组件化的架构设计，构建了具有高兼容性与可维护性的智能监控与预警平台。

在整体页面设计上，项目利用 CSS3 动画与渐变效果，营造出具有科技感的视觉体验，如发光效果和阴影扩散等。系统通过 WebSocket 全双工通信协议与后端 MySQL 数据库实现高效数据交互，确保模型推理结果的实时采集与展示。为满足电池数据监控与预警需求，系统集成了 ECharts 图表库，实现了多维度数据的直观呈现。为避免因数据线条过多而导致图形显示杂乱，系统内置了数据筛选功能，能够对实时采集及模型推理数据进行智能过滤，确保大屏数据可视化界面

既简洁又层次分明。初始页面展示图如下：

图 9 初始页面展示图

双击开始预测图如图 10，此时若是点击图表下方的图例，可以实现筛选功能，将不想看的指标过滤掉。

图 10 开始预测展示图

图 11 筛选掉电压 2 与温度 2 展示图

此外，系统实现了异步加载与按需加载机制，有效降低了页面初始加载负担，优化了数据请求流程，显著提升了页面加载性能。用户操作响应时延稳定控制在百毫秒至千毫秒区间，提升了整体用户体验并减少系统资源消耗。

2.1.3 故障列表与故障类型统计编号（MON-01-03）

当触发界面报警后，根据模型输出的故障分类结果，记录到故障列表，记录字段包括故障类型、发生时间和处理状态，同时为了能记录不在预设置的五种故障类型内的故障，我们支持人为的添加和删除故障类型。

具体实现如下：当模型检测到电池异常时，通过后端 API 将故障信息推送到前端，前端通过 WebSocket 实时接收报警信息，动态插入到故障列表表格中，系统自动标记新故障为"未处理"状态，并在界面显示醒目标识。在人工管理故障列表功能中，通过模态表单收集故障信息（类型下拉选择、时间选择器、状态单选），表单验证后通过本地存储 API 保存数据，删除功能上，每条记录配备删除按钮，操作后立即更新本地数据存储和界面显示，在状态标识上，使用不同颜色区分"已处理"（绿色）和"未处理"（红色）状态。故障列表暂时采用浏览器 localStorage 存储故障记录，键为 alarmData，数据格式为 JSON 数组，包含故障类型、格式化时间字符串和处理状态，页面加载时自动从本地存储初始化数据。

故障列表前端渲染如图 12 所示，点击删除可以删掉一整行的故障数据，人为添加故障类型功能如图 13 所示，输入内容框中支持用户任意输入故障类型，

时间框中支持用户选择发生故障的时间，第三个框为状态选择框，可以选择已处理与未处理。

图 12 故障告警列表示意图

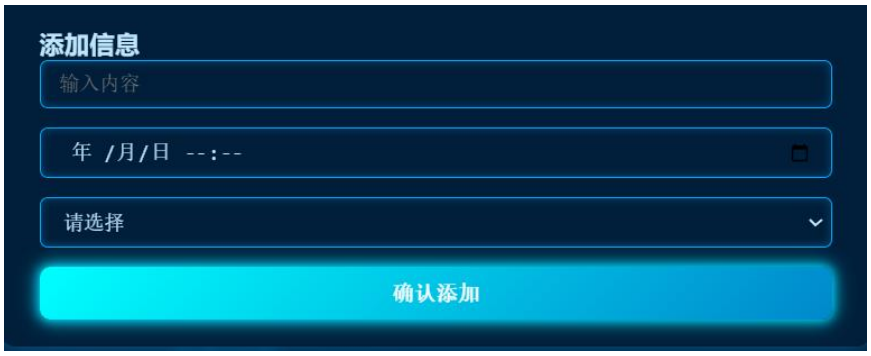

图 13 添加故障列表功能

在故障类型统计方面，我们实现了从告警列表中获取故障类型及发生频率。统计框中会基于收集的故障数据，按类型分类统计出现频次，使用 Chart.js 生成饼状图，直观展示各类故障占比。饼状图支持动态更新，当故障列表变化时自动重绘。 饼状图展示如图 14，当往告警列表中插入新故障时，饼状图会自动更新（如图 15）。

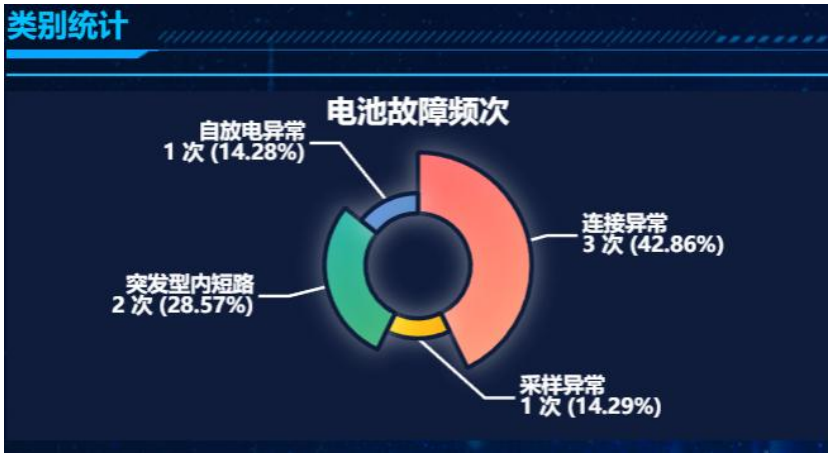

图 14 类别统计图

图 15 插入未知异常后更新饼状图

2.2 异常预警（MON-02）

当模型预测的电池状态超过预设的电池各监控字段阈值时，界面弹出红色预警弹窗进行警告，同时将此时模型的故障分类结果写入故障告警列表中。

2.2.1 阈值超限预警（MON-02-01）

根据已搭建好的数学模型，设置默认各监控字段的告警阈值，当模型预测的电池状态超过此时设置的阈值时，仪表盘周边显示脉冲式红色光晕特效，并弹出告警图表。

具体实现过程如下：系统预设电池监控字段的告警阈值（如电压阈值 4.2V，温度阈值 100℃），存储在后端数据库的 thresholds 表中。前端通过 setInterval 函数每秒从 /api/get_latest_data 接口中获取最新预测数据，与当前阈值比对。触发预警时前端显示脉冲式警告图标（warning 类元素），通过 CSS 动画实现呼吸光效，预警效果如图 16。

图 16 预警效果图

2.2.2 报警故障记录动态添加（MON-02-02）

模型根据预测数据异常模式（如电压突降、温度骤升）返回故障类型（如“绝缘失效”），将时间、故障类型等写入告警列表中，前端通过 localStorage 存储故障记录，并触发饼图更新。

改进 Transformer 模型检测到异常（如电压突降、温度骤升）后，后端 API 返回故障类型、发生时间及默认状态（未处理）。从浏览器 localStorage 读取 alarmData，将新故障信息（类型、格式化时间、状态）加入数组，将更新后的数组重新存入 localStorage（通过 saveAlarmToLocalStorage 实现）。通过 countFaultTypes 函数重新统计故障类型，然后使用 generateFailureChart 来更新饼图（如图 17）。

```javascript
function generateFailureChart() {
    // 先销毁旧图表实例
    const chartDom = document.getElementById('generateFailureChart');
    if (chartDom) {
        const existingChart = echarts.getInstanceByDom(chartDom);
        if (existingChart) {
            existingChart.dispose();
        }
    }

    // 创建新图表实例
    var myChart = echarts.init(document.getElementById('generateFailureChart'));

    // 从本地存储加载告警数据
    const alarmData = loadDataFromLocalStorage();

    // 统计每种故障类型的数量
    const countMap = {};
    alarmData.forEach(item => {
        if (countMap[item.type]) {
            countMap[item.type]++;
        } else {
            countMap[item.type] = 1;
        }
    });

    // 转换为 ECharts 需要的格式
    const failureData = Object.keys(countMap).map(type => ({
        name: type,
        value: countMap[type]
```

图 17 generateFailureChart 函数部分代码图

3. 阈值调整模块（THR）

3.1 阈值可视化配置（THR-01）

为了满足区域气候等环境因素的差异性会影响电池状态，因此我们设置了手动调整阈值界面，支持参数阈值滑动调节、点击调节以及手动输入，前两种方式（电压与温度为例）电压阈值可调节范围为 1-6V，步长为 0.1V，温度可调节范围是-100-150℃，步长为 1℃。

在代码实现上，通过 HTML 的<input type="range">滑块控件绑定数值输入框，滑块移动时实时同步显示数值。用户拖动滑块或直接输入数值均可调整阈值，输入时自动限制在有效范围内。同时在样式的选择上，在滑块轨道为深色渐变背景，背景上使用 Canvas 绘制动态粒子效果，粒子沿阈值变化方向流动，滑块按钮为发光蓝色圆形，悬停时放大并变为亮蓝色，增强交互反馈。调整好后设置了保存设置按钮，点击“保存”按钮时，通过 fetch 将阈值数据以 JSON 格式提交至后端接口/threshold 保存到 localStorage 中，下次可视化读取阈值时，只要超过该阈值便会弹出报警图层。调整阈值界面如图 18 所示。

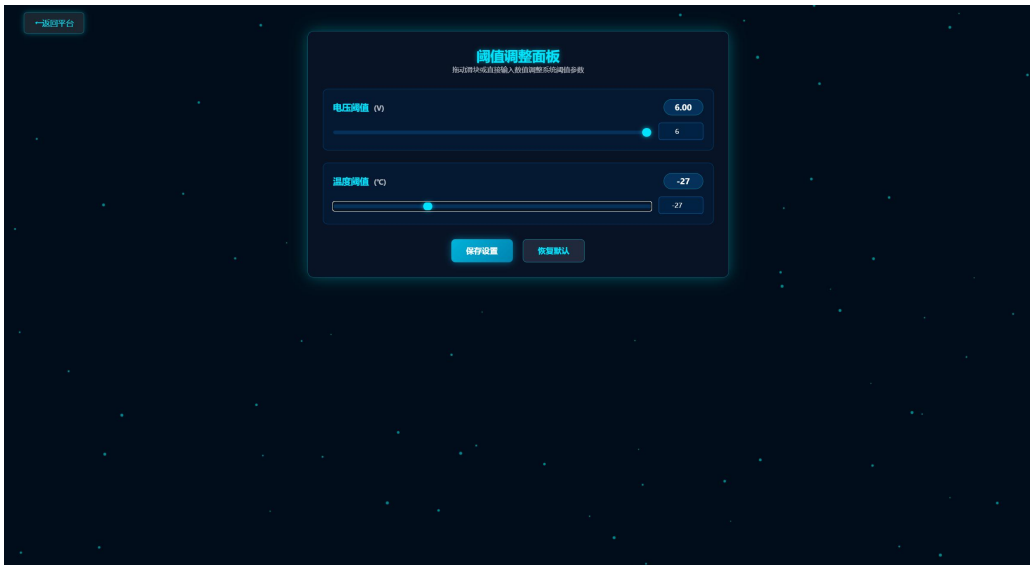

图 18 阈值调整界面

3.2 阈值方案快速回滚（THR-02）

若用户认为需要重新使用回默认阈值，我们提供恢复默认按钮，点击按钮即可迅速实现阈值方案的回滚。点击点击 “恢复默认” 按钮时，前端执行以下操作，重置滑块位置，将电压和温度的滑块控件动态调整至默认值对应的位置，随后同步输入框数值，关联的数值输入框内容立即更新为默认值，最后实时显示反馈，右侧的阈值显示区域（如 value-display）同步刷新为默认值，并通过视觉动画（闪烁）提示用户已重置。

4. 电池状态模块（BS）

4.1 电池状态可视化与续航数据（BS-01、BS-02）

该界面用于展示车主车辆的电池目前状态。环形图表显示当前电量百分比（0%-100%），颜色区分低/中/高电量区间（红/黄/绿）。仪表盘展示电池健康度（0%-100%），附带剩余寿命预估（如"7.2 年"）。充电速率盘中展示此时的充电速率。

在电量状态上，电量区间动态划分（0%-20%红色预警，20%-60%黄色提示，60%-100%绿色安全）。在 SOH 分析中，结合电池循环次数、内阻等参数计算健康度（未来考虑引入 RUL 模型进行寿命预测），整体界面上，采用渐变光效，外

环添加发光效果（box-shadow: 0 0 15px #00ff99），低电量时切换为红色脉冲动画，图表渲染使用 Canvas/SVG 混合模式，平衡流畅度与清晰度。电池状态界面如图 19。

图 19 电池状态界面

5. AI 咨询模块（AI）

5.1 智能对话咨询（AI-01）

为了在发生故障时第一时间为用户提供问题解答和诊断建议，我们通过以下方式实现了 AI 咨询功能。在前端界面设计上，使用科技感风格的聊天界面，包含粒子背景和扫描线效果，设计了消息气泡、输入框和发送按钮，支持实时对话交互，同时添加了打字机动画效果，模拟 AI 思考过程。部分前端设计代码如图 20 所示。

```javascript
// 处理 Markdown 中的 **bold** 与 "## 标题"
function mdFormat(text) {
    text = text.replace(/\*\*(.*?)\*\*/g, '<span class="bold">$1</span>');
    return text.split('\n').map(l => {
        if (l.startsWith('##')) return `<div class="heading">${l.slice(2).trim()}</div>`;
        return l;
    }).join('\n');
}

const chatBox : HTMLElement  = document.getElementById( elementId: 'chat-container');
const input : HTMLElement  = document.getElementById( elementId: 'chat-input');
const sendBtn : HTMLElement  = document.getElementById( elementId: 'send-btn');

// 打字机效果
function typeWriter(element, text, speed : number  = 30) : void  {
    let i : number  = 0;
    element.innerHTML = '';
    const timer : number  = setInterval( handler: () : void  => {
        if (i < text.length) {
            element.innerHTML += text.charAt(i);
            i++;
            chatBox.scrollTop = chatBox.scrollHeight;
        } else {
            clearInterval(timer);
        }
    }, speed);
}

function append(msg, who, typing : boolean  = false) : void  {
```

图 20 AI 咨询界面前端样式部分代码

在后端设计中，前端通过 EventSource 与后端建立长连接，实现流式响应，将用户问题实时发送到后端 API 接口，随后后端调用 DeepSeek 的 API 以流式（SSE 流式接口）获取 AI 生成的回答，前端实时显示接收到的回答内容。在特色功能上，使用 mdformat 函数，支持 Markdown 格式的粗体和标题渲染，同时，实现消息的渐显动画效果，添加了发送按钮的交互反馈效果，错误处理和连接失败提示。后端接口部分代码如图 21 所示。

```python
# ========= SSE 流式接口 =========
@app.get('/api/ai-stream')
def ai_stream():
    prompt = request.args.get('prompt','').strip()
    if not prompt:
        return Response( response: 'data: {"content": "**empty**"}\n\n', mimetype='text/event-stream')

    def generate():
        retries = 0
        while retries < 3:
            try:
                rsp = client.chat.completions.create(
                    model='deepseek-reasoner',
                    messages=[{"role":"system","content":"You are a helpful assistant."},
                              {"role":"user","content": prompt}],
                    stream=True
                )
                for chunk in rsp:
                    if chunk.choices[0].delta.content:
                        yield f'data: {json.dumps({"content": chunk.choices[0].delta.content})}\n\n'
                return
            except Exception as e:
                retries += 1
                time.sleep(1)
        yield f'data: {json.dumps({"content": "**AI 服务暂时不可用**"})}\n\n'

    return Response(generate(), mimetype='text/event-stream')
```

图 21 AI 咨询模块后端接口部分代码

最终通过 CSS 样式渲染出来的界面如所示。

图 22 AI 咨询模块界面示意图

6. 意见反馈模块（FB）

6.1 反馈提交（FB-01）

为了辅助产品迭代，集中管理反馈渠道，避免用户通过邮件或第三方平台提交的碎片化问题，建立用户与开发团队的透明沟通机制，提升产品信任度，其中，用户提供的意见均会存储到数据库中，意见反馈模块界面如图所示。

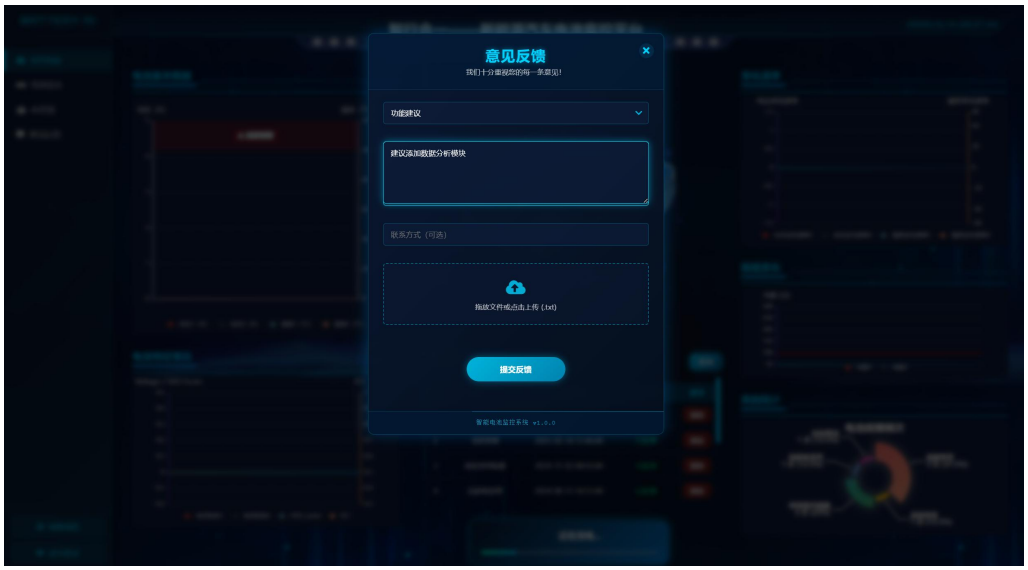

图 23 意见反馈模块界面

6.1.1 问题类型选择（FB-01-01）

用户可通过下拉菜单选择反馈类型（如功能建议、错误报告、界面问题等），帮助开发团队快速分类处理问题。在前端设计上，使用 <select> 元素创建下拉菜单，选项包括“功能建议”“错误报告”“界面问题”“其他问题”等。样式采用科技感设计：半透明背景、发光边框、动态箭头图标（如 Font Awesome 的 chevron-down），选中时触发高亮效果（如边框变为蓝色发光）。交互逻辑上，用户选择类型后，前端验证是否已选（未选时提示“请选择反馈类型”）。

6.1.2 详细描述输入（FB-01-02）

用户可在文本框中输入问题的详细描述，支持多行文本和格式化（加粗、换行）。前端设计上，使用 <textarea> 实现多行输入框，占位符提示“请输入您的详细反馈内容…”。样式上采用半透明背景、发光边框，聚焦时边框动画（流光效果）。交互逻辑上，实时统计字数，限制最大字符数（500 字）；输入内容为空时提交会提示“请填写反馈内容”。同时输入框带有扫描线动画，增强科技感。

6.1.3 文件附件上传（FB-01-03）

用户可上传文件，辅助说明问题，仅支持.txt 格式。前端设计上，在拖放区域中显示上传图标（cloud-upload-alt）和提示文字，支持点击选择和拖拽文件，上传后显示文件名，限制文件大小（5MB）。交互逻辑上，使用<input type="file">实现文件选择，通过 JavaScript 验证文件格式。非.txt 文件会提示“只能上传.txt 格式的文本文件”。样式上，拖放区域悬浮时显示高亮边框，上传成功后有绿色勾选动画。下图是上传 txt 文件后（图 24）的数据库展示图（图 25）。

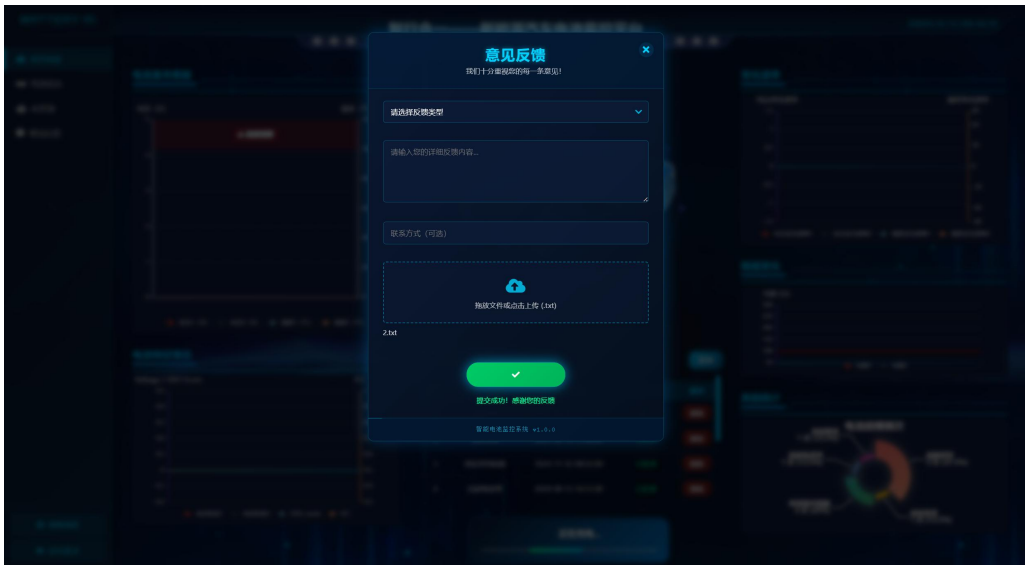

图 24 意见反馈界面上传 txt 文件示意图

Result Grid | Filter Rows: | Edit:

	id	account	message	feedback_time
▶	1	111	请加入数据库功能	2025-06-11 11:46:34
	2	111	请你记录意见	2025-06-11 11:48:27
	3	111	这	2025-06-11 11:51:33
	4	111	这是一条测试意见	2025-06-11 12:00:41
	5	13423276728	你好	2025-06-13 16:18:39
	6	111	建议添加数据分析模块	2025-06-14 20:46:01
	7	111	建议添加数据分析模块	2025-06-14 20:46:15
●	NULL	NULL	NULL	NULL

图 25 意见反馈数据库设计